logging systems is an unstructured string. Consequently, anything more than very simple analysis of the output is difficult. Ideally, you want to be able to query logs in the same way you can query the contents of a database. The solution to this problem is structured logging, which is supported by Serilog. *Structured logging* involves generating logs in a structured data format that enables advanced querying and filtering. In the final section of this chapter, you will learn more about this approach to logging and understand the benefits it brings.

12.1 Exception management

Exceptions are raised when a .NET application encounters an error condition. As a developer, you can choose to attempt to handle exceptions or not. If you don't handle an exception, your application stops execution of the current operation. What happens by default in a Razor Pages application at that point depends on whether the application is running in Development mode or not. This is determined by examining the application's `IWebHostEnvironment`, which we will look at in more detail in the final chapter when we explore environments. If the application is not running in Development mode, the default configuration results in `ExceptionHandlerMiddleware` intervening and executing the Error.cshtml page included in the initial project. This page is designed to provide no more than a bland message that an error has occurred. If the application is in Development mode, middleware invokes the *Developer Exception page*, which provides as much detail of the exception as possible.

12.1.1 The Developer Exception page

The purpose of the Developer Exception page is to help the developer to identify, locate, and fix any errors. Commonly, the Developer Exception